MATHLAB: A PROGRAM FOR ON-LINE MACHINE ASSISTANCE IN SYMBOLIC COMPUTATIONS

C. Engelman
Mitre Corporation
Bedford, Massachusetts

INTRODUCTION

The purpose of MATHLAB

A mathematical scientist experiments. Today, his test tube and his breadboard are blackboard and paper. He may, it is true, have available a computer, but its role is numerical and its results are delivered not today, not tomorrow, but the day after the final programming bug is corrected. The computer is not present during the most creative phases of the scientist's labor. The purpose of MATHLAB is to provide the scientist with computational aid of a much more intimate and liberating nature.

What sort of aid? The basic goal is to provide facilities for those operations which are mechanical. Among the most common of these are the addition of expressions and equations, the substitution of sub-expressions within a larger expression, differentiation, integration, Laplace transforms, multiplication of matrices, and the solution of simple equations. While the greater part of a scientist's time is spent on these mechanical pursuits (in fact, an appreciable portion is probably spent in simply checking answers and in the eternal bookkeeping problems of getting minus signs and 2π's right), we must keep in mind that most of the tedious computations associated with the creative aspects of his work are of a symbolic, rather than a numerical, nature. If we are to free the scientist from his routine mathematical chores and conserve his energies for the more properly human activities of interpretation, analysis, planning and conjecture, then we must mechanize the passage from r^2/r to r in addition to that from $1+1$ to 2.

Requisites for a Mathematical Laboratory

I should like to outline here the properties I feel are required of a mathematical laboratory, not in terms of the range of mathematical operations available, but rather in terms of its spirit and feel.

1. It should be capable of ordinary numerical computation. This implies the ability to perform arithmetic, to compute functions or to look up their values in tables, and to draw graphs.
2. It should be capable of a wide spectrum of symbolic computations.
3. The user commands should be simple. MATHLAB is intended for a physicist, not a programmer. The commands should be no more complicated than the user's thoughts. If he wishes to enter an equation

into the computer, he should need only to type the equation in a notation like that of ordinary mathematics. If he should then wish to differentiate that equation with respect to x, he should have to give a command no more complicated than "differentiate (x)."

4. It must be expandable by the expert. The language, functions, and subroutines of the laboratory must be such that it will grow as an organism. If today we write programs for symbolic differentiation, we should expect, tomorrow, to employ them in programs for power series expansions. The opportunity to expand the programs should be open to anyone who masters a well-defined and common computer language.
5. It should be extensible by the user. While the ability of the physicist to augment the existing programs will no doubt be severely limited compared to that of the programming expert, he should be provided tools for doing certain simple things for himself, such as changing notational conventions or teaching the machine the derivatives of his favorite functions.
6. The computer, as viewed by the user, must be intimate and immediate. The user should have next to his desk a console consisting of a typewriter or, preferably, a typewriter and a scope. Economy might, in some cases, dictate the substitution of a plotter for the scope. These are connected to a large, fast, on-line, time-shared digital computer. He communicates with that computer by typing messages on his typewriter or by means of a light pen on the scope. The computer replies by means of the same machines. It types both messages and equations. On the scope it displays both equation and graphs. Above all, the response time to the user's requests must be short.

Quick Summary of Existing Programs for Symbolic Mathematics

The first program we should mention was written for Whirlwind I by J. H. Laning and N. Zierler[1] and was not really a program for symbolic computation at all. But, even though it was capable only of numerical computation, it could accept programs written as simple symbolic mathematical expressions and perform them for a user with no machine language experience.

A later program that could accept instructions in the form of symbolic expressions, but also limited to numerical computations, was, of course, FORTRAN.[2]

Probably the most fundamental development, to date, for the adaptation of computers to symbolic computation is the design by J. McCarthy, for just such purposes, of a language called LISP (for LISt Processor).[3,4] Almost all of the symbolic computation programs extant are written in this language.

The foremost problem of content facing the constuction of a mathematical laboratory today is probably the writing of a satisfactory program for the simplification of mathematical expressions. Simplification is the unconscious of mathematics. We all simplify expressions every day, making choices appropriate to the occasion but of which we are almost totally unaware. For this reason, it is much simpler to program a more "advanced" formal computation, such as differentiation, which has exact rules of which we are conscious, than it is to have the machine simplify the answer after it completes the differentiation. We cannot afford to enter into a detailed discussion of the current attempts to solve this problem, but we would like to mention, in chronological order, the authors of three LISP programs for the simplification of mathematical expressions: T. Hart,[5] D. Wooldridge, Jr.,[6] and W. A. Martin.[7]

A LISP program for symbolic differentiation was written by K. Maling.[8] While it suffered from some weakness of simplification and from its input/output being restricted to well-formed LISP expressions (which are not nearly as legible as those written in ordinary mathematical notation), it was certainly a dramatic early demonstration of the ability of LISP to handle formal mathematical computation.

In my opinion, the most impressive example of symbolic mathematics yet performed by machine is the formal (or indefinite) integration program of J. Slagle written as a doctoral thesis under M. Minsky at MIT.[9] Perhaps the most interesting aspect of this program is that it was heuristic rather than algorithmic. It possessed a small table of integrals and

tried to reduce the problem to one or several found in that table by the same bag of tricks possessed by a good college freshman.

The remaining programs we shall discuss do not, like those above, represent attempts to perform particular symbolic processes (simplification, differentiation, integration) on a computer. They are, rather, attacks on the whole problem of building a system of symbolic computations.

The first such program we should like to mention is the mathematical laboratory project of W. A. Martin, a work-in-progress as a doctoral thesis at MIT under Professor M. Minsky. This work is by far the closest known relative to our MATHLAB. At present it appears that the main difference of emphasis will be that Martin's work will stress very broad input/output capabilities. He is, for example, working on input from scopes achieved by signifying with a light pen an interesting subexpression of a previously "printed" expression, as well as anticipating using the scopes for handwritten input. The emphasis in our program is more in the direction of continually increasing mathematical powers. Since both programs are written in LISP, a good deal of exchange should be possible.

Another approach to the mathematical laboratory problem is a project, under the direction of L. C. Clapp (while at Bolt), Beranek, and Newman.[10] The programs are not written in LISP but in a much simpler and weaker list-processing language of their own invention, called SIMPLIST. While this language and program might serve well for rapid performance of simple computations, e. g., adding two symbolic equations or evaluating a function entered symbolically, it seems unlikely that they will be capable of difficult symbolic procedures such as a powerful simplification program or symbolic integrations. These limitations stem primarily from the fact that their work is married to the PDP-1, a small computer.

The final system we should like to discuss before going on to our own MATHLAB is the IBM FORMAC system. This is an extension of FORTRAN which provides it with the ability to perform a certain amount of symbolic computation. It is under development in Cambridge under a group headed by J. E. Sammet.[11] It is quite capable, possessing such abilities as simplification, substitution, expansion of products of sums, factoring out power of a given variable, and differentiation. For a number of reasons, however, it does not seem well suited to a mathematical laboratory. It is program oriented. One does not give a simple command such as "differentiate" but rather one writes a program. True, the language, like FORTRAN, is easy for a nonprogrammer to learn, but it will not have the universal accessibility of, say, the simple commands of our MATHLAB. The programs can, of course, be run on-line. But they cannot be run, so to speak, line by line. It is necessary to write an entire program, for the program has to be compiled as an entity just like a FORTRAN program (in fact, it employs the FORTRAN IV compiler). It seems like a useful tool for symbolic computation when you know at the outset what you want to do, but quite inconvenient for experimentation.

MATHLAB—ON THE SURFACE

Mathlab is our current attempt at realizing a mathematical laboratory of the sort we have been discussing. The program, which has been developed on the time-shared system of PROJECT MAC at MIT and on the IBM 7030 at the MITRE Corporation, is continually growing, and the following description is accurate as of April 30, 1965. We do not feel that MATHLAB has, as yet, sufficient mathematical powers to be of aid to a general user, except with respect to special and occasional problems. It does, however, possess most of the qualities postulated in the previous section as requisities for a mathematical laboratory:

1. *Numerical computations.* It is very weak in this department because we decided at first to study symbolic computation as it represented the crux of our problem. It cannot draw graphs or evaluate common transcendental functions. We can evaluate algebraic expressions with numerical arguments in a variety of ways.
2. *Symbolic computations.* Here we can perform many common tasks. We can simplify, substitute, add equations, differentiate, integrate a little, solve equations, etc.
3. *The user commands are simple.* If the user has stored an equation called *e*1 and wishes to differentiate both sides of it with respect to x and call the resulting equation *e*2, he need only type:
   differentiate (el x e2).

4. The program can be expanded by any LISP programmer. In fact, we are doing this all the time.
5. MATHLAB can be extended a little by the user. He can teach the machine the derivatives of functions and change the names of system commands.
6. It is intimate. The user types in some initial equations; the computer thanks him; the user requests certain symbolic manipulations; the computer performs them and types back the answer; the user types in some expressions or numbers and requests the computer to substitute these for certain variables in a previous equation; the computer types back the answer; etc.

Mathematical Notation

In this section and the next we wish to describe MATHLAB as it appears to the user. There are some minor differences between MATHLAB as it exists at PROJECT MAC and on the STRETCH at MITRE. Where such differences occur, we shall describe the situation at MITRE. First, what sort of expressions may a user type to denote mathematical quantities? The answer is: those expressions, composed in the ordinary way, of the following entities:

1. Numbers: 1, 5/2, 2.5
2. Words, representing symbolic variables, composed of strings of letters and digits, the first of which is not a digit: x, distance, x1, x1sub2.
3. Operation symbols:
   + (addition)
   — (subtraction or minus)
   * (multiplication)
   / (division)
   ↑ (exponentiation)
4. Parentheses: "(" and ")". These have two functions. The first is to ensure the desired interpretation of certain expressions, e.g., to distinguish 5*(x+y) from 5*x+y. The second use of parentheses is for functional notation, e.g., sin(x).

All blanks are ignored. The rules of precedence of the operational symbols are conventional (FORTRAN) except that, in the absence of parentheses, e↑x↑2 denotes e↑(x↑2) not (e↑x)↑2 = e↑(2*x).

For example, if the user wishes to enter the mathematical expression (in conventional notation): $\sin(5x + y^2)$, he need only type (in MATHLAB's notation): sin(5*x + y↑2).

The System Commands

The program as we have developed it accepts two types of symbolic quantities, called *variables* and "equations," stored in the computer and which can be referred to and manipulated by "name." A variable is a mathematical expression together with its (one word) name. An equation is really two mathematical quantities, one for the left and the other for the right side of the equation, together with a (one word) name. The initial variable and equations in an experiment are entered into the computer by a function called "denote." For example, one might type:

```
denote nil
d = 1/2*a*t↑2,
e1 = = r↑2 = x↑2 + y↑2,
,
```

The first two commas signify the end of individual definitions and the third comma tells the computer that this is all the information we choose to give it for the time being. The effect of this input is to store in the computer a variable whose name is d and whose value is 1/2*a*t↑2 and an equation whose name is e1, whose left side is r↑2, and whose right side is x↑2+y↑2. From this point on the user never again has to type in r↑2 = x↑2+y↑2 but can simply refer to e1. Incidentally, the response to the above instruction (we shall, from now on, use the convention that we speak in lower case and the computer in capitals) is:

THANKS FOR THE VARIABLE D AND THE EQUATION E1.

In terms of these basic constructs of "variable" and "equation," we shall describe the various system commands:

repeat(x)

This repeats *x* to the user. *x* may be the name of either a variable or an equation. The format for a variable is similar to that of *denote*. For an equation, if *x* were *e*1 above, then "repeat" would print:

(E1) R↑2 = X↑2+Y↑2

This same format is used when any of the succeeding commands are called upon to print an equation.

please simplify(x y)

Simplifies x and names it y. In this, as in the following commands, the name "y" may be the same as the name "x." In this case, the old x is lost. If x is name of an equation, both sides are simplified independently.

forget(x)

does.

substitute((v1 v2 . . . vn) x y).

The first argument "(v1 . . . vn)" must be a list of names of *variables*. The *value* of each variable is substituted in x at each occurrence of its *name*. The new equation or variable (depending on x) is named y.

At this point we should like to state more precisely the meaning of the denote and substitute instructions. Should we give the command:

```
denote nil
z = x+y,
,
```

x and z would be quantities of quite different natures. We shall refer to "x" as a "formal symbol"; it is without meaning. "z," on the other hand, is the name of an official "variable"; its meaning, which we normally refer to as its "value," is the expression "x+y" constructed of the formal symbols "x" and "y." If we now type the instruction:

```
denotes nil
x = 5*t,
,
```

a variable whose name is "x" and whose value is 5*t would be created but this would in no way affect the status of x in the value x + y of the variable whose name is z. That x remains a formal symbol. The fundamental connection that can be established between these two occurrences (of different types) of the character "x" is through the instruction "substitute." If we now type:

substitute ((x) z w)

the program will look for a variable whose name is x, find that its value is 5*t, look for a variable whose name is z, discover that its value is x + y, substitute the expression 5*t (containing the formal symbol t) for all occasions of the formal symbol x in x + y (obtaining the expression 5*t + y), simplify this to 5*t + y, and create a new variable whose name is w and whose value is 5*t + y.

Substitutions take place only upon command, never automatically. This is as it should be. The user may have previously informed the computer that x = r*cos(t), but he might like to type x without having it automatically changed to r*cos(t). Automatic substitution schemes are not only undesirable, but prone to interminable loops.

This description may seem too detailed but an understanding of the distinction between a variable and a formal symbol as well as the function of the substitute instruction is fundamental to an understanding of MATHLAB. We shall presume the extension of these concepts to equations and to other commands (e.g., differentiate) are apparent.

add((q1 q2 . . . qn)name)

The q's can be equations, variables, or numbers in any order. If there is at least one equation among them, *name* is an equation; otherwise, it is a variable. Equations are added by adding left sides and adding right sides independently. Variables or numbers are added to an equation by adding them to both sides of the equation.

multiply((q1 . . . qn name) . . .

Similar to above.

subtract(x y name) . . .

Similar to above, but only two equations, variables, or numbers are subtracted instead of an indefinite number as in add and multiply.

division(x y name) . . .

Similar to above.

raise(x y name) . . .

Similar to above. "name = x^y"

negative(x y) . . .

"y = –x"

invert(x y) . . .

"y = 1/x"

flip(x y) . . .

x must be the name of an equation. y becomes the name of that equation with the left and right sides interchanged. This is useful if we wish, say, to add the left side of one equation to the right side of another.

makeequation(x y) . . .

x must be the name of a variable. An equation is formed whose name is y, whose left side is the name "x," and whose right side is the value of x. For example, using the variable "d" of "denote" above, if the instruction makeequation (d e2) were given, the computer would respond:

(E2) D = 1/2*A*T↑2

makevariable(e) . . .

e must be an equation whose left side is a single word. Then a variable is formed whose name is the left side of e and whose value is the right side of e. For example, "makevariable(e2)" would now produce:

D = 1/2*A*T↑2

which is where we started.

expand((x1 x2 . . . xn)) . . .

This produces no immediate result but affects all succeeding simplifications. Whenever one of the x's (which are formal symbols) occurs in a product of sums, that product is multiplied out. The following dialogue will clarify this.

```
denote nil
e3 = = y↑3 + y*y = (x + z) * (u + y),
,
THANKS FOR THE EQUATIONS E3
pleasesimplify(e3 e4)
(E4) Y↑3+Y↑2 = (X+Z)*(U+V)
expand((x u))
YES
pleasesimplify(e3 e5)
(E5) Y↑3 + Y↑2 = X*U+X*V+Z*U+Z*V
```

The "expand" command affects not only the command "pleasesimplify" but other commands such as "substitute" or "add," which always simplify their answers.

factor((x y . . .)) . . .

Like "expand," this produces no immediate result but affects all future simplifications. It causes the collection of all terms containing, as a factor, a power of x and similarly for y. The order of formal symbols in the list "(x y . . .)" implies precedence. In this case, the term "x*y" will count as y occurrences of x rather than x occurrences of y.

We give an example:

```
denote
mitre = a*x+2*x+x*y+y
  +b*y+4*x↑2+c*x↑2,
,
THANKS FOR THE VARIABLE MITRE.
pleasesimplify(mitre bedford)
BEDFORD = A*X+2*X+X*Y+Y
  +B*Y+4*X↑2+C*X↑2
factor ((x y))
YES
pleasesimplify(mitre bedford)
BEDFORD = (2+A+Y)*X+
  (4+C)*X↑2+(1+B)*Y
```

By calling the functions expand and factor at different times with different arguments, the user can maintain control over the form of his answers.

differentiate(y x yprime) . . .

Differentiates y with respect to x and calls the resulting variable or equation (depending on y) yprime. At present, all differentiation is explicit.

learnderivative . . .

Allows the user to teach the computer the derivative of a new function. The format of this command is best explained by an example:

```
learnderivative nil
arctan,
x,
1/(1 +x↑2),
```

The need for this function has, to some extent, been obviated by an improvement in the differentiation program. If the computer is asked to differentiate arctan(x)↕2, it will decide that the answer is twice arctan(x) times the derivative of arctan(x). If it should then discover that it does not know the derivative of arctan, it will try to obtain it from the typewriter. If it succeeds, it will complete the differentiation and remember the derivative of arctan for future use.

integrate(v x w) . . .

v must be a variable which is a rational function of x with (rational) numerical coefficients.

w is its indefinite integral. For a more precise discussion, see the following section. We give an example. If

```
v = (x+1)/((x↑2+1)*(x↑2+
        x+1)↑3),
```

then the command

"integrate(v x w)"

yields the result:

```
W = (2/3*X↑3+1/2*X↑2+1/3*X—1/2)
    /(X↕4+2*X↑3+2*X↑2
    +2*X+1)+1/2*LOG
    (X↑2+1) — ARCTAN(X) —
    1/2*LOG(X↑2+X+1)+7/(3*SQRT(3))
    *ARCTAN((2*X+1)/SQRT(3))
```

solve(e x) . . .

e must be an equation which is equivalent to a rational function (with symbolic coefficients) of x. At present it can handle only those equations which are really (although not necessarily explicitly) quadratic or linear in x. The reply to this command, excepting those cases which the program cannot solve, takes one of three forms depending on whether the computer analyzes the equation to be linear, quadratic with distinct roots, or quadratic with a double root. The following three examples illustrate these different responses.

(1) If e is the equation: y = a*x+b, then "solve(e x)" yields:

```
THIS EQUATION HAS A SINGLE ROOT.
X = (Y — B)/A
```

(2) If e is the equation: a*x↑2+b*x+c = 0, then "solve(e x)" yields:

```
THIS EQUATION HAS TWO ROOTS,
  NAMED FIRSTROOT AND SECOND
  ROOT.
FIRSTROOT = 1/2*( — B+SQRT(B↑2 —
  4*A*C))/A
SECONDROOT = 1/2*( — B — SQRT(B↑2
  — 4*A*C))/A
PLEASE RENAME THOSE WHOSE PRES-
  ERVATION YOU WISH TO ENSURE.
```

(3) If e is the equation: x↑2 — 2*b*x+b↑2 = 0, then "solve(e x)" yields:

```
THIS EQUATION HAS A DOUBLE ROOT.
X = B
```

We give a fourth example which, though simple, exhibits more of the meat of the program:

(4) If e is the equation: 1/(x↑2 — 1) = 1/(x — 1), then "solve(e x)" yields:

```
THIS EQUATION HAS A SINGLE ROOT.
X = 0
```

There are two points here. First, the equation does not appear, at first glance, to be linear but the program analyzes it as such. Second, a naive attempt at solving the equation, e.g., inverting both sides, could yield what we feel is an extraneous root at x = 1. For an explanation of how the program avoids this trap, see the next section.

rename(x y) . . .

The variable or equation that had the old name x obtains the new name y.

newname(A B) . . .

This command differs from all previous commands in that it affects, not the data, but the system commands themselves. It creates a new system command B whose effect is identical to A and which exists in addition to A. For example, if the user tires of typing "differentiate," he can give the command

"newname(differentiate d)"

after which the command

"d(y x yprime)"

will have exactly the same effect as

"differentiate(y x yprime)."

Before digging beneath the surface of MATHLAB, it might help clarify some of the preceding if we give a very short sample session possible today.

```
denote nil
e1 = r↑2 = s*t,
s = x↑2 * y,
t = log(w)/x,
,
THANK YOU FOR THE VARIABLES S T
  AND THE EQUATION E1.
substitute ( (s t) e1 e2)
(E2) R↑2 = X*Y*LOG(W)
denote nil
w = sin(x↑2+y↑2),
,
```

```
THANK YOU FOR THE VARIABLE W.

substitute ((w) e2 e3)
(E3) R↑2 = X*Y*LOG(SIN(X↑2+Y↑2))
differentiate(e3 x e3prime)
(E3PRIME) 0 = Y*LOG(SIN(X↕2+Y↕2))
  + 2*X↑2*Y*COS(X↑2 + Y↑2)/SIN(X↑2
  + Y↑2)
denote nil
,
THANKS FOR NOTHING
```

This last crack on the computer's part is indicative of a fact that we have not mentioned heretofore. Most of the commands have very heavy error protection. If the user makes a mistake in constructing a variable or an equation or tries to give a variable a name already assigned to an equation, etc., he will receive an instructive error return.

MATHLAB — BELOW THE SURFACE

LISP

The entire program is written in LISP.[4] The system commands are all addressed to the LISP evalquote operator, e.g., the command "differentiate(h t dh)" presents the evalquote operator with a couple consisting of the function "differentiate" and the list of arguments (h t dh)."

Numbers

The LISP system, written by R. Silver and P. Heckel[12] for the IBM 7030 (STRETCH) at MITRE, contains only one type of number, namely, rational numbers, i.e., ordered pairs of integers. If any of the numbers 12/5, 24/10, or 2.4 is typed in, it is converted to the rational 12/5.

Internal Representation of Mathematical Expressions

The internal representation of any mathematical expression is a well-formed LISP S-Expression in a prefix notation. If the user types:

```
denote nil
v = t↑2+sin(pi*t),
,
```

then there is stored on the property list of the atom "v" the property "variable" followed by the S-Expression:

(PLUS (EXPT T 2)(SIN(TIMES PI T))).

Should the user then type:

```
differentiate(v t dv)
```

then there is stored, upon completion of the differentiation and simplification, on the property list of the atom "dv" a pointer to the S-Expression:

(PLUS(TIMES 2 T)(TIMES PI (COS(TIMES PI T)))).

This is translated back into the original infix notation and the typewriter prints:

DV = 2*T + PI*COS(PI*T)

Equations are stored similarly to variables, except there are two pointers, one to the S-Expression representing the left side of the equation and one to the right.

Besides the obvious need for well-formed LISP expressions, there are two reasons for our choice of this internal representation of mathematical expressions. First, this representation has become fairly standard and this allows us to exchange programs with other workers. Second, the prefix notation turns out to be well suited to our applications. Consider the differentiation we have just discussed. About the easiest and fastest thing for LISP to tell us about any list is what is the first item on it, in this case: "PLUS." But this is precisely the first thing our differentiation program would want to know so as to invoke the rule that the derivative of a sum is the sum of the derivatives. Both the input (infix→prefix) and the output (prefix→infix) translation programs are written in LISP, the former employing the character reading functions.

Other internal representations of expressions also occur, e.g., polynomials as lists of coefficients and rational functions as dotted pairs of lists of coefficients. All such alternative representations have translation programs connecting them in both directions with the standard prefix representation.

Simplification and Differentiation

The internal programs for simplification and differentiation have been borrowed from the Stanford Simplify Program.[6] They have been modified in two ways.

Simplify has been enlarged to handle a family of simplifications typified by the transformation: (EXPT (MINUS X) 4/3)→(EXPT X 4/3), i.e., $(-x)^{4/3} \rightarrow x^{4/3}$. This simplification was impossible in the original Stanford system because 4/3 could only be represented by an approximating decimal such as 1.3333333 and nothing can be done with $(-x)^{1.3333333}$.

Differentiation has been modified so as to look to the typewriter for the derivatives of new functions.

Integration

The program for the integration of rational functions with numerical coefficients was written by M. Manove at MITRE in the summer of 1964.[13] It is based on a theorem of Hardy[14] that states that the integral of such a function is of the form $R_1 + \int R_2$ where R_1 and R_2 are rational and R_2 has only simple poles. The program always finds R_1 and does the best it can with $\int R_2$, that best depending on its ability to factor the denominator of R_2. It is good enough to have found several errors in published tables of integrals.

Solve

Solve first brings the equation it has to solve over on one side. It then combines the various terms into a single rational expression with one numerator and one denominator, employing greatest common divisor routines to eliminate common factors from numerators and denominators. The roots of the original equation are then the roots of the numerator of the constructed rational function. If that numerator is quadratic, its roots are found by the quadratic formula.

REFERENCES

1. J. H. Laning and N. Zierler, "A Program for Translation of Mathematical Equations for Whirlwind I," Engineering Memorandum E-364, Instrumentation Laboratory, MIT (Jan. 154).

2. *Reference Manual, 709/7090 FORTRAN Programming System,* IBM, 1958.

3. J. McCarthy, "Recursive Functions of Symbolic Expressions and Their Computation by Machine," Communications of the ACM, Apr. 1960.

4. *LISP 1.5 Programmer's Manual,* MIT Computation Center and Research Laboratory of Electronics, Aug. 1962.

5. T. Hart, "SIMPLIFY, Artificial Intelligence Project," RLE and MIT Computation Center, Memo 27 (n.d.).

6. D. Wooldridge, Jr., "An Algebraic SIMPLIFY Program in LISP," Stanford Artifical Intelligence Project, Memo No. 11 (Dec. 27, 1963).

7. W. A. Martin, "Hash-Coding Functions of a Complex Variable," MIT Project MAC, Artifical Intelligence Project, Memo 70, Memorandum MAC-M-165 (June 25, 1964).

8. K. Maling, "The LISP Differentiation Demonstration Program," Artificial Intelligence Project, RLE and MIT Computation Center, Memo 10 (n.d.).

9. J. R. Slagle, "A Heuristic Program that Solves Integration Problems in Freshman Calculus, Symbolic Automatic Integrator (SAINT)", MIT Mathematics Department, Ph.D. thesis, 1961.

10. L. C. Clapp and R. Y. Kain, "A Computer Aid for Symbolic Mathematics," *Proceedings of the Fall Joint Computer Conference,* 1963.

11. J. E. Sammet and E. R. Bond, "Introduction to FORMAC," *IEEE Transactions on Electronic Computers,* Aug. 1964.

12. P. Heckel, forthcoming MITRE Technical Report.

13. M. Manove, "INTEGRATE: A Program for the Machine Computation of the Indefinite Integral of Rational Functions," MITRE Technical Memorandum, TM-04204.

14. G. H. Hardy, *The Integration of Functions of a Single Variable,* Cambridge University Press, 1958.